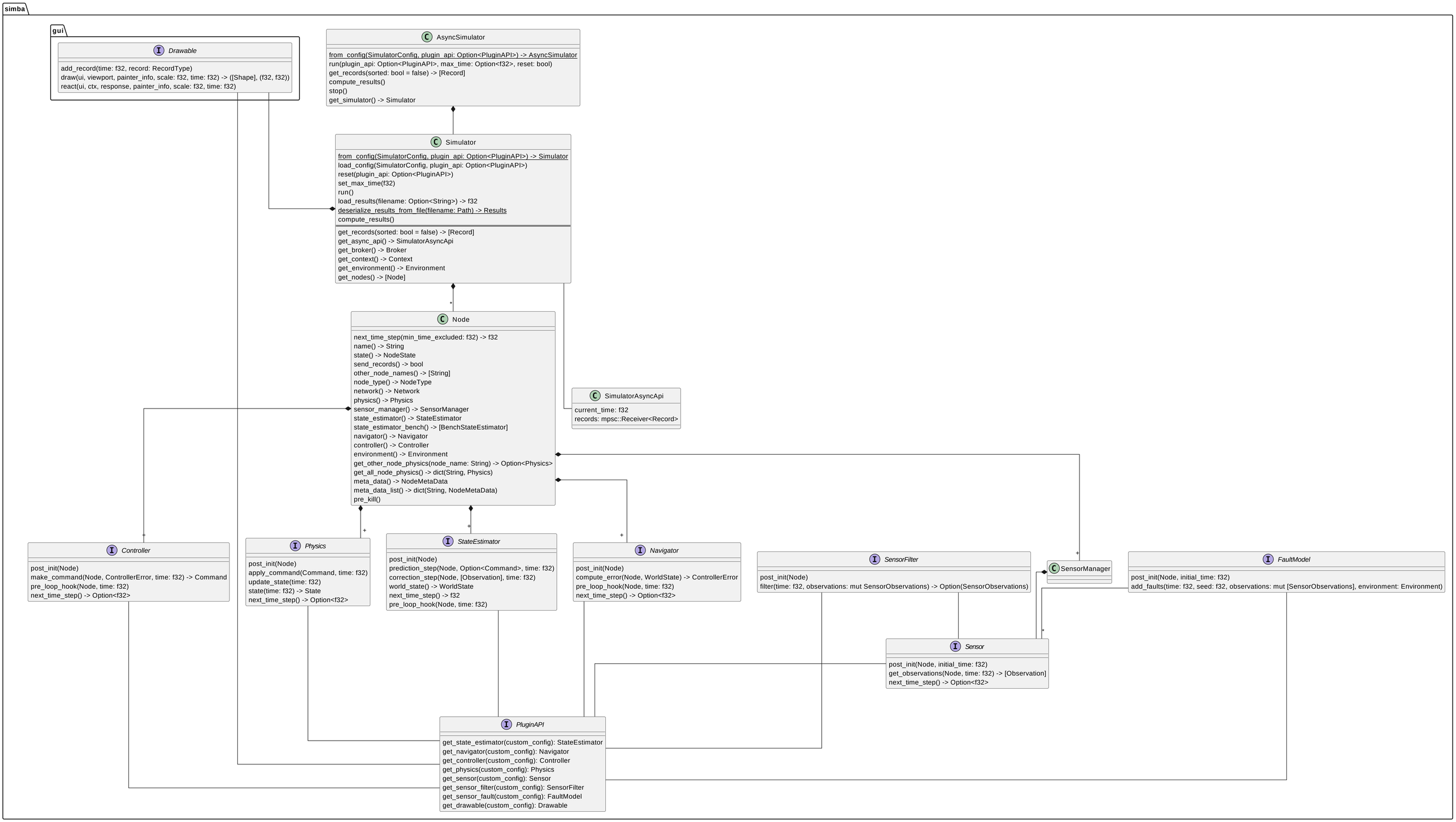
simba
gui
Drawable
add_record(time: f32, record: RecordType)
draw(ui, viewport, painter_info, scale: f32, time: f32) -> ([Shape], (f32, f32))
react(ui, ctx, response, painter_info, scale: f32, time: f32)
AsyncSimulator
from_config(SimulatorConfig, plugin_api: Option<PluginAPI>) -> AsyncSimulator
run(plugin_api: Option<PluginAPI>, max_time: Option<f32>, reset: bool)
get_records(sorted: bool = false) -> [Record]
compute_results()
stop()
get_simulator() -> Simulator
Simulator
from_config(SimulatorConfig, plugin_api: Option<PluginAPI>) -> Simulator
load_config(SimulatorConfig, plugin_api: Option<PluginAPI>)
reset(plugin_api: Option<PluginAPI>)
set_max_time(f32)
run()
load_results(filename: Option<String>) -> f32
deserialize_results_from_file(filename: Path) -> Results
compute_results()
get_records(sorted: bool = false) -> [Record]
get_async_api() -> SimulatorAsyncApi
get_broker() -> Broker
get_context() -> Context
get_environment() -> Environment
get_nodes() -> [Node]
Node
next_time_step(min_time_excluded: f32) -> f32
name() -> String
state() -> NodeState
send_records() -> bool
other_node_names() -> [String]
node_type() -> NodeType
network() -> Network
physics() -> Physics
sensor_manager() -> SensorManager
state_estimator() -> StateEstimator
state_estimator_bench() -> [BenchStateEstimator]
navigator() -> Navigator
controller() -> Controller
environment() -> Environment
get_other_node_physics(node_name: String) -> Option<Physics>
get_all_node_physics() -> dict(String, Physics)
meta_data() -> NodeMetaData
meta_data_list() -> dict(String, NodeMetaData)
pre_kill()
SimulatorAsyncApi
current_time: f32
records: mpsc::Receiver<Record>
Controller
post_init(Node)
make_command(Node, ControllerError, time: f32) -> Command
pre_loop_hook(Node, time: f32)
next_time_step() -> Option<f32>
Physics
post_init(Node)
apply_command(Command, time: f32)
update_state(time: f32)
state(time: f32) -> State
next_time_step() -> Option<f32>
StateEstimator
post_init(Node)
prediction_step(Node, Option<Command>, time: f32)
correction_step(Node, [Observation], time: f32)
world_state() -> WorldState
next_time_step() -> f32
pre_loop_hook(Node, time: f32)
Navigator
post_init(Node)
compute_error(Node, WorldState) -> ControllerError
pre_loop_hook(Node, time: f32)
next_time_step() -> Option<f32>
SensorFilter
post_init(Node)
filter(time: f32, observations: mut SensorObservations) -> Option(SensorObservations)
SensorManager
FaultModel
post_init(Node, initial_time: f32)
add_faults(time: f32, seed: f32, observations: mut [SensorObservations], environment: Environment)
Sensor
post_init(Node, initial_time: f32)
get_observations(Node, time: f32) -> [Observation]
next_time_step() -> Option<f32>
PluginAPI
get_state_estimator(custom_config): StateEstimator
get_navigator(custom_config): Navigator
get_controller(custom_config): Controller
get_physics(custom_config): Physics
get_sensor(custom_config): Sensor
get_sensor_filter(custom_config): SensorFilter
get_sensor_fault(custom_config): FaultModel
get_drawable(custom_config): Drawable